

CMSC451: Algorithms

JAMES ZHANG^{*}

October 7, 2024

These are my notes for UMD's CMSC451: Algorithms. These notes are taken live in class ("live- $\text{\TeX}$ "-ed). This course is taught by Professor Andrew Childs.

Contents

1	Introduction	2
1.1	Stable Matching Problem	2
1.2	Asymptotic Notation	3
1.3	Basic Graph Algorithms	4
1.3.1	Graph Representations	4
1.3.2	Paths, Cycles, and Connectivity	4
1.3.3	Trees	5
1.3.4	More on Connectivity	5
1.3.5	Connectivity in Digraphs	7
1.3.6	Topological Sorting	7
2	Algorithm Design Techniques	9
2.1	Greedy Algorithms	9
2.1.1	Shortest Paths	10
2.1.2	Minimum Spanning Trees	10
2.1.3	Interval Scheduling	11
2.1.4	Interval Partitioning	12
2.1.5	Scheduling to Minimize Lateness	13
2.2	Divide and Conquer	14
2.2.1	Closest Points	14
2.2.2	Polynomial Multiplication and the Fast Fourier Transform	14

^{*}Email: jzhang72@terpmail.umd.edu

Definition 0.1 (Algorithm). An **algorithm** is a well-defined computational procedure for processing some input to obtain a desired output.

§1 Introduction

§1.1 Stable Matching Problem

Definition 1.1 (Stable Matching Problem). The goal is to pair members of two sets according to their preferences, such as students and employees for summer internships. Given two sets S, J a **matching** is a set of pairs of the form $(s, t). s \in S, t \in J$, such that every element of S appears in at most one pair, and similarly for J .

Note 1.2. For simplicity, the two sets have the same size, and we must match every member of the first set with exactly one member of the second set.

Definition 1.3. A matching is **perfect** if every element of S and J is in some pair.

Definition 1.4. A matching is **stable** if there is no pair (s, t) not in the matching such that s prefers t over its assigned partner and t prefers s over its assigned partner.

Example 1.5

See August 29. notes for some examples.

Definition 1.6. Gale-Shapley Algorithm

```

1 All companies and students are unengaged
2 While there is a company c that is unengaged and has not made an offer to
  every students
3   Let s be the highest-ranked student for c to whom c has not yet made an
    offer
4   If s is unengaged
5     (c, s) become engaged
6   Else
7     Let c' be the company to which s is engaged
8     if s prefers c to c' then
9       (c,s) become engaged
10      c' becomes unengaged
11    endif
12  endif
13 endwhile
14 Return the set of engaged pairs

```

Lemma 1.7

Once a student receives an offer, they remain engaged for the rest of the algorithm, and the quality of their engagement (according to their preference) can only increase.

Proof. Omitted. □

Lemma 1.8

As the algorithm proceeds, the sequence of students engaged to a given company can only decrease in performance.

Proof. Omitted. □

Lemma 1.9

If there are n companies and n students, then the algorithm terminates after at most n^2 iterations of the while loop.

Proof. Omitted. □

Lemma 1.10

The Gale-Shapley algorithm outputs a perfect matching.

Proof. Omitted. □

Theorem 1.11

The matching output by the G-S algorithm is stable.

Proof. Omitted. □

§1.2 Asymptotic Notation

We're interested in asymptotic upper bounds on the worst-case running times of algorithms, and we express this using **Big-O Notation**

Definition 1.12. $f(n) = \mathcal{O}(g(n))$ if $\exists c > 0, n_0 \geq 0$ such that for all $n \geq n_0$, $f(n) \leq cg(n)$

Definition 1.13. $f(n) \in \Omega(g(n))$ if $\exists c > 0, n_0 \geq 0$ such that for all $n \geq n_0$, $f(n) \geq cg(n)$.

Example 1.14
 $10n^2 \in \mathcal{O}(n^2), \mathcal{O}(n^3), \Omega(n^2), \Omega(n), \dots$

Definition 1.15. We say $f(n) \in \Theta(g(n))$ if $f(n) \in \mathcal{O}(g(n))$ and $f(n) \in \Omega(g(n))$

Definition 1.16. We say $f(n) \in o(g(n))$ if $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = 0$ and $f(n) \in \omega(g(n))$ if $\lim_{n \rightarrow \infty} \frac{g(n)}{f(n)} = 0$

§1.3 Basic Graph Algorithms

Definition 1.17. A **directed (undirected)** graph $G = (V, E)$ is a finite set V of vertices and E of edges, where edges are **ordered (unordered)** pairs of vertices.

Definition 1.18. Vertices are sometimes called **nodes**

Definition 1.19. Edge $\{u, v\}$ joins u and v , which are its **ends**. u and v are **adjacent**; the edge is **incident** in u and v . Edge $\{u, v\}$ has tail u and head v

Definition 1.20. The **degree** of vertex v is the number of incident edges, in a digraph, $\text{indeg}(v)$ is the number of edges with v as head, and vice versa for $\text{outdeg}(v)$.

Note 1.21. We often write $|V| = n$ and $|E| = m$. Furthermore, note

$$\sum_{v \in V} \deg(v) = 2m \quad \sum_{v \in V} \text{indeg}(v) = m = \sum_{v \in V} \text{outdeg}(v) \quad m \leq \binom{n}{2} = \mathcal{O}(n^2)$$

in an undirected graph. In a directed graph, $m \leq n^2$.

Definition 1.22. A **subgraph** of $G = (V, E)$ is a graph $G' = (V', E')$ with $V' \subseteq V$ and $E' \subseteq E$

§1.3.1 Graph Representations

Definition 1.23. An **adjacency matrix** is a $|V| \times |V|$ matrix indexed by vertices, with $A_{uv} = 1$ if $(u, v) \in E$ and 0 otherwise.

Definition 1.24. A **adjacency list**: for each $v \in V$, given a list Adj_v of its neighbors outgoing

§1.3.2 Paths, Cycles, and Connectivity

Definition 1.25. A **path** is a sequence of vertices $(v_0, \dots, v_k)$ such that $(v_i, v_{i+1}) \in E$ for all $i \in \{0, 1, \dots, k-1\}$. A path is **simple** if all of its vertices and edges are distinct. The length is the its number of edges.

Definition 1.26. A **cycle** is a path with $v_0 = v_k$ and all other vertices are distinct.

Definition 1.27. The **distance** between $u, v \in V$ is the length of the shortest path from u to v .

Definition 1.28. An undirected graph is **connected** if for all $u, v \in V$, $\exists$ a path from u to v . A **component** of a graph is a set of vertices such that pairs of vertices in the set have a path between them, and no superset has this property.

Definition 1.29. A digraph is **strongly connected** if for all $u, v \in V$, $\exists$ a path from u to v and a path from v to u .

Definition 1.30. A **strongly connected component** is a maximal subgraph that is strongly connected.

§1.3.3 Trees

Definition 1.31. A **forest** is a graph with no cycles, and a **tree** is a connected forest. Note that a **n-vertex** tree has $n - 1$ edges. If we designate one vertex of the tree as a root, then the unique neighbor of a vertex that is closer to the root is the parent, and the neighbors that are further from the root are its children. Vertices of degree 1 in a tree are called **leaves**.

Definition 1.32. A digraph with no cycles is called a **Directed Acyclic Graph** or **DAG**.

§1.3.4 More on Connectivity

Given a graph, we want to find if it is connected? Given vertices s, t is there a path between them ($s - t$) connectivity?

One approach is to find a **spanning tree**, a subgraph that includes all vertices and is a tree.

Solution. We can specify a rooted tree by giving the parent $\text{pr}[v]$ of every vertex with $\text{pr}[\text{root}] = \emptyset$. Here is the pseudocode for generating a spanning tree for a graph rooted at s .

```

1  Let T be the graph with one vertex, s, with  $\text{pr}[s] = \text{emptyset}$ .
2  While exists an edge  $\{u, v\}$  joining a vertex u of T to a vertex v not
   in T
3    Add vertex v and edge  $\{u, v\}$  to T
4    Set  $\text{pr}[v] = u$ 

```

□

The resulting graph is always a tree.

Lemma 1.33

The algorithm produces a spanning tree for the component of s .

Proof. Any vertex in the tree has a path to s : repeatedly take parents until you reach s . To find a path from u to v in T , join the paths from u to s and v to s . On the other hand, if u is not in T , then there is no path to s . If there were, we could follow the path and find an edge from a vertex not in T to a vertex in T , but no such edge remains when the algorithm is done. □

Definition 1.34. Breadth-First Search (BFS): explore the graph in order of increasing distance from the root.

```

1  Let T be the tree with one vertex, s, with pr[s] = empty
2  Repeat:
3      Let u be the unexhausted vertex that joined the tree earliest, the
        "active vertex"
4      Choose an edge {u, v} where v is not in T
5      Add vertex v and edge {u, v} to T
6      Set pr[v] = u
7  Until all vertices are exhausted

```

The total running time of BFS is $\mathcal{O}(\sum_{u \in V} (\deg(u) + 1)) = \mathcal{O}(m + n)$

Lemma 1.35

BFS partitions the vertices into layers L_j . A vertex in L_j has distance j from s .

Proof. By induction on j . Clearly, $L_0 = \{s\}$ is the set of vertices at distance 0 from s . If $L_0, L_1, \dots, L_j$ satisfies the lemma, then vertices in L_{j+1} must have distance at least $j + 1$ from s . But there exists a path of length $j + 1$ from any vertex in L_{j+1} to s since there is an edge to a vertex in L_j , which has a path of length j to s by IJ. Hence, vertices in L_{j+1} satisfies the lemma, so the lemma follows by induction. $\square$

Definition 1.36. A graph $G = (V, E)$ is **bipartite** if there is a partition (A, B) of V where $V = A \cup B$, $A \cap B = \emptyset$, such that for all $\{u, v\} \in E$, either $u \in A$ and $v \in B$ or $u \in B$ and $v \in A$.

Lemma 1.37

Let T be a BFS tree of graph G , and let u, v be vertices of T with $u \in L_i$ and $v \in L_j$. Suppose $\{u, v\}$ is an edge of G . Then $|i - j| \leq 1$.

Proof. WLOG, suppose u is added before v . Then $i \leq j$ and u is active before v . There are two cases:

1. v is in the tree when u becomes active. Then $\{u, v\}$ is a non-tree edge, and $\text{pr}[v]$ joined the tree before u , so $j = \text{layer}(\text{pr}(v)) + 1 \leq i + 1$.
2. v is not in the tree when u becomes active. Then $\{u, v\}$ is a tree edge and $j = i + 1$.

$\square$

Theorem 1.38

A connected graph with BFS tree T is bipartite if and only if there is no non-tree edge joining vertices in the same layer.

Proof. Suppose there is no non-tree edge joining vertices in the same layer. Let $A = L_0 \cup L_2 \cup \dots \cup L_{2k}$ and let $B = L_1 \cup L_3 \cup \dots \cup L_{2k+1}$. This is a partition since all non-tree edges join vertices whose layers differ by 1.

Conversely, suppose $\{u, v\}$ is a non-tree edge between vertices in the same layer. Suppose their nearest common ancestor is j layers higher, then the path in the tree between u and v has length $2j$. Adding $\{u, v\}$ gives a cycle of length $2j + 1$, but an odd cycle cannot be bipartite. $\square$

§1.3.5 Connectivity in Digraphs

We can use BFS to find the vertices that can be reached from a given starting vertex. The idea is to run BFS from every vertex, so there are n runs of *BFS*.

Note 1.39. The digraph is **strongly connected** if and only if all runs reach all vertices, and the running time would be $\mathcal{O}((m + n)n)$

Lemma 1.40

If u and v are mutually reachable, and v and w are mutually reachable, then u and w are mutually reachable.

Proof. We can go from $u \rightarrow w$ by going from $u \rightarrow v \rightarrow w$ and we can go from $w \rightarrow u$ by going from $w \rightarrow v \rightarrow u$. $\square$

The better idea is the following algorithm

-
- 1 Run a BFS starting from s
 - 2 Run BFS starting from s , reversing the directions of all edges
 - 3 If both searches find all vertices, the graph is strongly connected, otherwise not
-

The running time of this is $\mathcal{O}(2(m + n)) = \mathcal{O}(m + n)$

§1.3.6 Topological Sorting

The idea here is that we can use a digraph to capture a set of operations with constraints on which ones must be done before others.

Definition 1.41. A **topological ordering** is a possible sequence in which the actions can be performed, subject to the constraints on which other actions must have been completed.

Lemma 1.42

If G has a topological ordering, then G is a DAG.

Proof. For a contradiction, suppose G has a topological ordering and a directed cycle C , $v_1, v_2, \dots, v_n$. Let v_i be the lowest-index vertex on C , and let v_j be the vertex before it in C . Then (v_j, v_i) is an edge with $i < j$, but we also must have $j < i$ for this to be a topological ordering. This is a contradiction. $\square$

Lemma 1.43

Every DAG has a vertex with indegree 0.

Proof. We proceed by proving the contrapositive: if every vertex has positive indegree, then the graph is a directed cycle. Start at any vertex. Walk along some edge in the backward direction, which must be possible because every vertex has positive indegree. Repeat. After at most $n + 1$ steps, we must have visited some vertex twice, so we found a cycle. $\square$

Lemma 1.44

Every DAG has a topological ordering.

Proof. By induction on $n = |V|$. Base case, a 1-vertex graph has a topological ordering. Suppose the lemma holds for any digraph with at most n vertices. For an $(n + 1)$ -vertex graph, let v be a vertex with indegree 0. Put it first in the ordering. Follow it by a topological ordering of $G/\{v\}$. This exists because $G/\{v\}$ is obtained by deleting a vertex and edges from a DAG, and deletion cannot guarantee a cycle, and plus the IH, and we have shown a valid topological ordering. $\square$

Theorem 1.45

Using the above Lemmas, we have shown that a graph is a DAG if and only if it has a topological ordering.

```

1 TopoSort(G):
2   Let S be an emptyset
3   For all vertices v
4     If v has indegree 0, add v to S
5     Set count[v] = indeg(v)
6   endfor
7   while S is nonempty do
8     remove a vertex v from S
9     output v
10    for each vertex u that v points to
11      remove edge (v, u) from the graph
12      decrement count[u]
13      if count[u] = 0, then add u to s
14    endfor
15  endwhile

```

There is $\mathcal{O}(n)$ initialization, the while loop goes through all vertices; for loop for v takes $\mathcal{O}(\text{outdeg}(v) + 1)$ steps $\implies \mathcal{O}(n + \sum_v (\text{outdeg}(v) + 1)) = \mathcal{O}(m + n)$

§2 Algorithm Design Techniques

§2.1 Greedy Algorithms

A common problem for greedy algorithms is the coin change problem. Fix the coin denominations $c_1, c_2, \dots, c_k$, and given a value v , find a way to make up v using as few coins as possible.

```

1 Change(v):
2   If v = 0, return 0
3   Else
4     Let c be the largest coin of denomination at most v
5     Add c to the list of coins
6     Change(v-c)
7   endif

```

The question that we want to ask is, "does this find the maximum number of coins?"

Note 2.1. If the coin denominations are 1, 7, 10 and $v = 14$, as the greedy algorithm would say $10 + 1 + 1 + 1 + 1 = 14 \implies 5$ but the optimal is actually $7 + 7 = 14 \implies 2$

§2.1.1 Shortest Paths

Given weights $w_{uv} \geq 0$ for each edge $(u, v) \in E$, what is the length of a shortest path from s to t , for $s, t \in V$?

Definition 2.2. Dijkstra's Algorithm finds the shortest path in a digraph with no negative edge weights.

```

1  Let d[s] = 0 and pr[s] = empty
2  For all v in V \ {s}, let d[v] = infinity
3  Mark all vertices as unexplored
4  Repeat:
5    Let u be the unexplored vertex with smallest d[u]
6    For each neighbor v of u
7      If d[u] + w_{uv} < d[v] then
8        Let d[v] = d[u] + w_{uv}
9        Let pr[v] = u
10   endif
11   endfor
12   Mark u as explored
13 Until all vertices are explored

```

Lemma 2.3

When v is explored, $d[v]$ is the distance from s to v .

Proof. By induction on the number of explored vertices, base case, this clearly holds when s is explored. IH: suppose this lemma holds when k vertices are explored. Let v be the $(k+1)$ st explored vertex; let $\text{pr}[v] = u$. By IH, the algorithm has already found the shortest path from $s \rightarrow u$. Suppose for a contradiction that the shortest path from s to v does not go through u . Let x be the last explored vertex on the path and let y be its unexplored neighbor on this path. Then

$$\text{dist}(s, v) < d[v] \leq d[y] = \text{length of path from } s \text{ to } y \text{ found in alg} \leq \text{dist}(s, v)$$

which is a contradiction. □

§2.1.2 Minimum Spanning Trees

Goal: given a graph GG with edge weights, find a spanning tree T that minimizes the total weight of edges in T

Definition 2.4. A **cut** in a graph is a bipartition of the vertices ($V = A \cup B$, $A \cap B = \emptyset$). A cut is **nontrivial** if neither A nor B is empty. An edge **crosses** the cut if it has one vertex on each side.

Definition 2.5. Kruskal's Algorithm adds the lowest-cost edge that doesn't create a cycle in order to construct a MST.

Lemma 2.6

Assume all edge weights are distinct. Let (A, B) be a nontrivial cut; let e be the minimum weight crossing this cut. Then any minimum spanning tree must contain e .

Proof. Let T' be a MST that does not contain $\{u, v\}$. Since T' is spanning, there exists a unique path in T' from u to v . Since u and v are on opposite sides of the cut, then there must be an edge along this path that crosses the cut; call it e' . Let T be the graph obtained from T' by deleting T' and adding e . We can show that T is a spanning tree. Since e is the min-weight edge crossing the cut, $w_e < w_{e'}$, and since all other edges are unchanged, T has lower total weight than T' . $\square$

Theorem 2.7

Kruskal's algorithm outputs a MST.

Proof. Suppose the algorithm adds edge $e = \{u, v\}$ to the forest F . Consider the cut with $A =$ vertices in the component of u in F . Vertex v is on the other side of the cut since adding e doesn't create a cycle. There are no edges in F crossing the cut, so e is in fact the min-weight edge crossing the cut, so e is in fact the min-weight edge crossing the cut. So, by the lemma, every MST contains e . $\square$

Definition 2.8. The idea of **Prim's Algorithm** is to choose any root and grow a tree by repeatedly adding the non-tree vertex with the lowest attachment cost.

Theorem 2.9

Prim's Algorithm outputs a MST.

Proof. Suppose the algorithm adds $e = \{u, v\}$ in the forest F . Consider the cut with $A =$ the component of the root in F . Clearly, e crosses the cut, and it has the lowest weight of any such edge by the greedy rule, so by the lemma, every MST contains e . The algorithm outputs a spanning tree since as long as F is not connected, we can add some edge and still have no cycles. $\square$

Note 2.10. Both of these algorithms can be implemented in $\mathcal{O}(m \log n)$ with appropriate data structures.

§2.1.3 Interval Scheduling

In this course, we cover three greedy problems with respect to interval scheduling

1. finding the largest possible compatible subsets
2. minimizing the resources used

3. minimizing lateness

Given n requests with starting times s_i and finishing times f_i for $i \in \{1, \dots, n\}$. A subset of requests is compatible if no two overlap in time. Find the largest possible subset.

Note 2.11. When we think about how we want to be greedy, we want to **choose the earliest-finishing activity first!**

```

1 GreedyIntervalSchedule(s, f):
2   Sort tasks by increasing order of finish time
3   Let A be an empty set
4   Let f_prev = -inf
5   For i = 1 to n:
6     if s_i > f_prev
7       Add task i to A
8       f_prev = f_i
9   endif
10  endfor

```

The running time of this algorithm is $\mathcal{O}(n \log n)$. Note that the scheduling contains no conflicts because we only add intervals that start after the last finished interval.

Theorem 2.12

GreedyIntervalSchedule outputs an optimal solution.

Proof. Let $G = (g_1, \dots, g_k)$ be the greedy schedule (with indices of intervals in order of increasing finishing time). Let $B = (b_1, \dots, b_l)$ be an optimal schedule. Let j be the first index where the schedules differ. Thus,

$$G = (g_1, \dots, g_{j-1}, g_j, \dots, g_k) \quad B = (g_1, \dots, g_{j-1}, b_j, \dots, b_l)$$

By the greedy rule, $f_{g_j} \leq f_{b_j}$. So, let's switch B to the valid schedule

$$B' = (g_1, \dots, g_{j-1}, b_{j+1}, \dots, b_l)$$

Repeating, we can get another valid schedule

$$B'' = (g_1, \dots, g_k, b_{k+1}, \dots, b_l)$$

Since the greedy algorithm stopped with only k jobs, there cannot be a job b_{k+1} that it could have scheduled, so in fact $k = l$ and the greedy schedule is optimal. $\square$

§2.1.4 Interval Partitioning

2. Now suppose we need to schedule all the intervals and we want to minimize the resources used.

Definition 2.13. The **depth** of a set of intervals is

$$\max_t |\{i \in \{1, \dots, n\} : t \in [s_i, f_i)\}|$$

Clearly the number of resources is at least the depth.

```

1 GreedyIntervalPartition(s, f):
2   Sort the intervals by starting time
3   For i = 1 to n:
4     Let E = emptyset
5     For j = 1 to i - 1:
6       If [sj, fj] overlaps [si, fi] then add color[j] to E
7     endfor
8     Let color[i] be the smallest positive integer that is not in E
9   endfor

```

Theorem 2.14

Using GreedyIntervalPartition, each interval is assigned a color from $\{1, 2, \dots, \text{depth}\}$ and no two overlapping intervals are the same color.

Proof. Wherever we assign a color, it is different from the colors of all overlapping intervals, so we never assign the same color to two overlapping intervals. Now we need to show that we don't exclude all colors from $\{1, 2, \dots, \text{depth}\}$. Say that when considering interval i , it overlaps t previous intervals. Then $\exists t + 1$ overlapping intervals, so $\text{depth} \geq t + 1 \implies t \leq \text{depth} - 1$, so there is always a color available. $\square$

§2.1.5 Scheduling to Minimize Lateness

3. Now consider the problem where we are scheduling to minimize lateness. Each task has a duration t_i and a deadline d_i . Define the **lateness** of task i , if scheduled in $[s_i, f_i] = [s_i, s_i + t_i]$ as $l_i = \max\{0, f_i - d_i\}$ and our goal is to minimize

$$\max_i l_i$$

the worst-case lateness.

Note 2.15. Here, the strategy is to schedule in order of increasing deadline d_i .

Definition 2.16. A schedule has an **inversion** if job i comes before job j but $d_i > d_j$.

Lemma 2.17

All schedules with no inversions and no idle time have the same max lateness.

Proof. The only flexibility in such a schedule is in the order of jobs with the same deadline. Any such jobs must be scheduled consecutively. The finishing time of the last of these jobs doesn't depend on the order. $\square$

Lemma 2.18

There is an optimal schedule with no inversions and no idle time.

Proof.

□

§2.2 Divide and Conquer

Note 2.19. First, recall mergesort: to sort a list of length n , we must divide the list into two halves and sort each of them recursively, and then merge the two lists in time $\mathcal{O}(n)$. The recurrence relation can be modeled as

$$T(n) = 2T\left(\frac{n}{2}\right) + \mathcal{O}(n) \implies T(n) = \mathcal{O}(n \log n)$$

by the Master's Theorem

§2.2.1 Closest Points

Given n points in the plane, find the closest pair. A simple approach finds such a pair in $\mathcal{O}(n^2)$ time.

The main idea of the better algorithm is to recursively divide into left and right halves, find the closest pair of points on each side, check whether there is a closer pair of points with one on the left and one on the right. As we recursive, maintain sorted lists by x and y coordinates.

Let us have input points $P = \{p_1, \dots, p_n\}$ where each $p_i = (x_i, y_i)$.

$Q = \text{pts in } P \text{ w first } n/2 \text{ coords}$ $R = \text{pts in } P \text{ with last } n/2 \text{ coords}$

Find the closest pair of points in $Q = (q_0^*, q_1^*)$ and $R = (r_0^*, r_1^*)$. Let $\delta = \min(\text{dist}(q_0^*, q_1^*), \text{dist}(r_0^*, r_1^*))$ and let x^* be the largest x coordinate of a point in Q . Let L be the line $x = x^*$.

Lemma 2.20

If $\exists q \in Q, r \in R$ with $\text{dist}(q, r) < \delta$, then both q, r must be within distance δ of L .

Lemma 2.21

If two points in S have distance less than δ then they must be within 15 positions of each other in the list of points sorted by y coordinate.

Proof. Divide the strip into sequences of side length $\frac{\delta}{2}$.

□

§2.2.2 Polynomial Multiplication and the Fast Fourier Transform

Problem: given polynomials $a(x) = a_0 + a_1x + a_2x^2 + \dots + a_{n-1}x^{n-1}$ and $b(x) = b_0 + b_1x + b_2x^2 + \dots + b_{n-1}x^{n-1}$ compute their product

$$c(x) =$$